

Normative Rule-Centric Knowledge Graph Construction

omitted for blind review^{1,*†}

¹omitted for blind review)

Abstract

Normative documents exhibit a well-defined logical structure but high semantic density, characterized by recurring terminology, cross-references, and hierarchical rule dependencies. These characteristics pose significant challenges for AI systems, which often struggle to retrieve factually grounded and contextually accurate information from such texts. Despite the growing body of research on Legal Knowledge Graphs, limited attention has been given to structure-oriented KG construction explicitly focused on extracting normative rules from canonical legal elements. Existing approaches frequently emphasize entity extraction or case-based reasoning, rather than modeling the internal normative architecture of the document itself. This paper proposes a structure-aware methodology for constructing Knowledge Graphs from normative documents within the Civil Law tradition. Our approach leverages the inherent normative hierarchy—Articles, Caput, Paragraphs, Items, and Sub-items—to extract and formalize rules as semantically cohesive RDF triples. We define a method for generating structured subgraphs that, when combined, form a coherent and interpretable Knowledge Graph representation of the normative corpus. To validate the methodology, we selected a normative document with high semantic similarity and developed a dataset of 700 evaluation questions, including 20 deliberately ambiguous or socially engineered queries. The evaluation assesses the graph’s ability to preserve structural dependencies and improve factual grounding in information retrieval tasks.

Keywords

legal documents, knowledge graph construction, large language models,

1. Introduction

Normative documents such as regulations, legal texts, and contracts are foundational instruments for governance across multiple domains. These documents exhibit a predefined structural organization rooted in the legal tradition of their country of origin, including Civil Law (Roman–Germanic) and Common Law systems. In the Civil Law tradition, which is the focus of this work, normative texts are hierarchically structured into Articles, Caput, Paragraphs, Items, and Sub-items (álíneas), forming a layered and interdependent rule system.

Despite this structural regularity, normative documents present substantial computational challenges. They define obligations, permissions, prohibitions, and sanctions that interact through complementarity, hierarchy, exception, and precedence (e.g., the caput establishing the general rule, while subordinate clauses specify or restrict its scope). Moreover, such documents often contain high semantic density, recurring terminology, cross-references, and temporal amendments. These characteristics make it difficult for AI systems to retrieve factually grounded and structurally consistent information.

Recent advances in Retrieval-Augmented Generation (RAG) have improved textual retrieval through vector similarity. However, purely vector-based approaches do not preserve normative hierarchy, rule dependencies, or temporal validity. As a result, systems may retrieve semantically similar passages that do not correspond to the governing rule, limiting explainability and legal reliability.

To address these limitations, we propose a structure-aware methodology for constructing Knowledge Graphs directly from normative documents. Our central hypothesis is that explicitly encoding normative hierarchy and rule relationships during graph construction improves interpretability, factual grounding, and resistance to ambiguity in downstream AI applications.

Woodstock’22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY

*Corresponding author.

✉ email@email.com (o. f. b. review)

🆔 0000-0000-0000-0000 (o. f. b. review)



© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

This paper contributes: (i) a methodology for transforming normative documents into RDF-based Knowledge Graphs; (ii) a domain ontology tailored to normative modeling; (iii) a constrained LLM-based extraction and ingestion strategy that preserves ontology fidelity; and (iv) a dynamic evaluation benchmark to assess factual consistency and logical coherence..

2. Related Work

The application of Large Language Models (LLMs) to Knowledge Graph (KG) construction has been extensively investigated. Zhu et al. [1] provide a comprehensive evaluation of LLMs for KG construction and reasoning, demonstrating that while some models show strong performance in reasoning tasks, they exhibit limited reliability as few-shot information extractors. Their experiments indicate that LLMs perform better as reasoning assistants than as primary extractors, highlighting the need for approaches that leverage this specialization. Our work extends this perspective by demonstrating that, in the context of normative documents, advanced reasoning-capable LLMs can extract not only entities and relations, but also complex prescriptive rules when guided by a predefined ontology that encodes the structural organization of normative documents.

The GraphRAG framework, proposed by Edge et al. [2], represents a significant advancement over purely vector-based RAG approaches for large-scale sensemaking tasks. By constructing an entity–relationship graph, followed by hierarchical community detection and multi-level summary generation, GraphRAG enables comprehensive and diverse global responses to queries requiring cross-document understanding. The authors show that this graph-based approach substantially outperforms vector RAG in terms of comprehensiveness and diversity. However, its generic implementation does not exploit the formal structure inherent to specialized domains such as legal and normative texts.

For legal and normative domains, de Martim [3] proposes an adaptation of GraphRAG that explicitly models the hierarchical structure inherent in normative documents. His approach introduces a formal model inspired by FRBRoo (Functional Requirements for Bibliographic Records), distinguishing between: 1 Norms (abstract works), 2 Hierarchical components (titles, chapters, articles), 3 Temporal Versions (the text as it exists at specific points in time), and 4 Language Versions. By modeling normative evolution as precise aggregations of modified components rather than redundant compositions, the author enables deterministic and context-aware retrieval of legal texts at any historical point in time.

Identified Gap and Contribution Although these works establish strong foundations for KG construction in formal domains, they primarily focus on extracting entities and relations or modeling document-level structure. A critical gap remains in the identification, extraction, and explicit representation of prescriptive rules, the normative statements that define obligations, permissions, and prohibitions within legal and regulatory documents.

Unlike general knowledge extraction, rule extraction requires understanding complex linguistic patterns, normative expressions (e.g., obligations, permissions, prohibitions), and the interplay between rule components (conditions, subjects, actions) and the entities they govern.

Our work addresses this gap by extending the GraphRAG paradigm to model rules as first-class citizens within the knowledge graph. We introduce a specialized Rule node class that captures not only the structural and temporal context of normative provisions, but also generates a textual representation of the rule’s semantics and explicitly connects it to relevant entities, conditions, and governed objects. This rule-centric modeling approach enables more granular reasoning over regulatory compliance and normative interpretation, going beyond the capabilities of existing KG construction methods for normative documents.

3. Methodology

This section presents a sequential and dependency-driven methodology for constructing RDF triples (subject–predicate–object) from normative documents. Each phase builds upon the previous one,

forming a cumulative modeling pipeline in which conceptual abstraction precedes formalization, which in turn constrains extraction.

The process is structured into five interdependent phases: 1. Domain scoping, 2. Ontology design and formalization, 3. Normative-aware textual segmentation, 4. Ontology-constrained LLM triple extraction, 5. RDF ingestion.

3.1. Domain scoping

This is a pre-ontological phase that establishes the conceptual boundaries of the domain, following the scope definition and glossary construction steps proposed in [4]. Without this abstraction layer, ontology construction would lack semantic grounding, and triple extraction would degenerate into uncontrolled relation generation. This is a three-step analytical process:

1. Comparative analysis of segmented normative corpora Conducted across distinct normative segments rather than treating regulatory texts as a homogeneous corpus. For instance: technical manuals tend to encode procedural obligations and operational constraints, internal regulations formalize behavioral rules and governance structures.

2. Identification of recurrent semantic patterns Beyond surface linguistic variation, we identified recurring prescriptive structures (obligations, permissions, prohibitions), actor-role configurations, institutional references, and structural hierarchies.

3. Conceptual clustering of structural elements Recurring elements were grouped into five primary conceptual clusters: Core Entities (objects regulated by the norm, e.g., equipment, spaces, resources), Agents and Roles (individuals or collectives assuming normative responsibilities), External Institutions (regulatory bodies, oversight entities, third parties), Normative Artifacts (documents, rules, articles, clauses), Auxiliary Contextual Elements (temporal, conditional, procedural modifiers).

While the detailed analysis focused on condominium regulations and conventions, high-level inspection of other normative genres revealed recurring structural patterns, supporting the identification of a shared conceptual backbone across normative discourse.

3.2. Ontology design and formalization

For the condominium governance domain, we define a set of top-level ontological axes that categorize entities and operate as semantic constraints for class instantiation. These axes restrict the admissible class space during triple extraction. In the prompt design, the LLM is explicitly instructed to instantiate only predefined classes and properties, prohibiting the creation of new ontological elements.

- PhysicalStructure (Classes such as :Building, :Unit, :CommonArea);
- PeopleAndRoles (Classes such as :CondominiumManager, :Resident, :Doorman);
- ExternalInstitutions (Classes such as :Supplier, :InsuranceCompany);
- DocumentsAndRules (Classes such as :InternalRegulation, :Fine);
- OperationalActivities (Classes such as :Maintenance, :Reservation).

This schema guides the typing of nodes (Subjects and Objects) in RDF triples.

Predicate selection is governed by a semantic control layer termed *Intention*. Intention captures the pragmatic function of the text fragment within normative discourse and determines which property (predicate) will connect the nodes in the graph.

Three primary intentions are defined for this domain; other normative domains may require a different number or type of intentions:

- Regulatory: Encodes permissions, prohibitions, and obligations (e.g., :permittedIn, :prohibitedIf, :appliesTo);
- Descriptive: Encodes structural or attributive relations (e.g., :hasCommonArea, :hasPurpose);
- Factual/Record: Encodes temporally grounded events (e.g., :occurredOn, :reservedBy).

By separating node typing (ontological axes) from predicate selection (intention), we introduce a dual-layer control mechanism that constrains generative behavior at extraction time. This design mitigates the need for extensive human prompt refinement and reduces the risk of hallucinated or structurally inconsistent outputs, limitations widely discussed in LLM-based KG construction [1].

The T-Box (terminological component) is manually constructed by a human domain expert prior to any automated extraction. All classes and properties are defined a priori and remain fixed throughout the process. The LLM is restricted to populating the A-Box under these closed-schema constraints, with no permission to modify or extend the ontology.

3.3. Normative-aware textual segmentation

A normative document in a Civil Law system is structured hierarchically and textually, typically composed of Articles, Caput, Paragraphs (§), and Items (I, II, III). The *caput* is the main text of a normative article, appearing before any paragraphs or subdivisions. It contains the core legal rule of the article and serves as the interpretative basis for all subsequent provisions.

For this reason, we use the caput as the main node in each subgraph. A unique identifier is assigned to the caput, and this identifier is propagated to all child elements through a `parent_id` parameter.

At this stage, the document is fully structured, with the caput acting as the main axis and being connected to each component of the article. An article may consist solely of a caput or include multiple paragraphs, all of which are linked through a shared parent identifier.

Once normalized, the document is segmented in a way that preserves its semantic integrity. Unlike generic recursive character splitters (e.g., fixed chunk sizes of 1,000 characters with 200-character overlap), our approach ensures that each structural element (article or paragraph) retains its complete original text exactly as it appears in the source document.

In parallel, we generate an additional chunk-based representation within the same file. Complete structural elements (articles or paragraphs) are appended sequentially until an approximate size of 900 characters is reached. Importantly, no article or paragraph is ever split across chunks. Each segment is included in full, even if this slightly exceeds the target size. This strategy preserves both semantic and structural coherence.

The resulting structured representation, preserves both hierarchical metadata and textual content, and serves as the input for the ontology-constrained extraction phase.

3.4. Ontology-constrained LLM triple extraction

With the ontology in place, each structured representation (i.e., each caput and its associated elements) is processed by a large language model. Rather than performing unconstrained triple extraction, the model operates under a rigid schema defined by a closed set of allowed classes and properties, as described earlier.

For each caput, the model extracts candidate RDF-style triples, explicitly associating every entity and relation with the originating structured representation identifier (`parent_id`). For example, if a caput states that motorcycles are not allowed to park in a designated location, the model produces a triple such as:

$$\text{Motorcycle} \rightarrow \text{:notPermittedIn} \rightarrow \text{FirstFloorParkingLot}$$

This approach ensures structural consistency, traceability, and compatibility with downstream graph reasoning.

The extracted triples are first converted into an intermediate `GraphDocument` representation using the `LLMGraphTransformer` library. Each `GraphDocument` is then transformed into an RDF/Turtle file using the `rdflib` library. The serialization enforces explicit types, namespaces, and property definitions aligned with the T-Box.

Rather than performing generic entity extraction, the LLM is guided to identify the normative actors, regulated objects, and conditions that constitute each rule, explicitly capturing deontic relations (permissions, prohibitions, and obligations) as typed predicates constrained by the T-Box.

These constraints are enforced through a structured prompt that provides the LLM with the complete T-Box definition, the segmented text, and explicit instructions disallowing the generation of classes or properties outside the predefined schema. The LLM is therefore restricted to populating the A-Box without modifying the underlying ontology.

Beyond extracting explicit relational triples, the LLM is also instructed to instantiate a `:Rule` class element linked to the caput through the same `parent_id`. The normative rule, even when implicit or distributed across multiple clauses in the original text is synthesized by the LLM and stored in the `description` field of this element. This representation serves both as a human-readable summary of the normative content and as an additional typed node available for downstream graph reasoning.

This design enables SPARQL queries to retrieve, for any given rule, both its textual formulation and the entities to which it is connected. The resulting context rule text plus linked entities, is then made available to the LLM retriever as grounded evidence for question answering, ensuring that responses are anchored to explicit normative content.

3.5. RDF ingestion

The final output of the pipeline consists of two synchronized knowledge representations: (i) a vector index (vector database) that supports semantic similarity retrieval, and (ii) a Knowledge Graph (triple store database) that encodes juridical structure, rule applicability, and the relationships between documents, entities, and roles. Because both layers share the same `parent_id`, they can be jointly queried in a hybrid fashion, enabling the system to retrieve not only the most relevant textual evidence but also the structural and juridical context necessary for explainable answers.

The RDF ingestion process occurs in two stages. In the first stage, the main subgraph corresponding to the caput is inserted, including all associated nodes (entities, roles, and the `:Rule` instance). In the second stage, all paragraphs are inserted as instances of the `:Paragraph` class and connected as child nodes of the corresponding caput entity through the shared `parent_id`. This staged insertion preserves hierarchical integrity and ensures consistency between structural and semantic representations.

3.6. Running Example

To concretely illustrate the proposed rule-centric construction process, we introduce a running example based on a simplified excerpt of a condominium regulation. Consider the following provision:

Art. 12. *It is prohibited to park motorcycles in the first-floor parking lot. §1. The restriction applies to both residents and visitors. §2. Motorcycles may be parked only in the designated external parking area.*

This excerpt is representative of normative discourse in Civil Law systems: a caput establishing a primary rule, followed by paragraphs that qualify its scope and conditions. Although linguistically compact, it encodes multiple structural and semantic components, including regulated objects (motorcycles), spatial constraints (first-floor parking lot and external parking area), actor scope (residents and visitors), and deontic modality (a prohibition combined with a restricted permission).

In the initial abstraction stage (1. Domain Scoping), the excerpt is interpreted beyond its surface wording. “Motorcycle” is identified as a core regulated entity, parking areas are categorized as physical structures, residents and visitors are conceptualized as normative roles, and the article itself is classified as a normative artifact. More importantly, the semantic pattern of a prohibition with scoped applicability is recognized as a recurring prescriptive structure within the domain.

These abstractions are then constrained by a predefined ontology (2. Ontology Design and Formalization). Classes such as *Motorcycle*, *ParkingLot*, *Resident*, *Visitor*, and *Rule* are explicitly declared in the T-Box, together with properties encoding normative intentions, such as *notPermittedIn*, *textitappliesTo*, and *allowedOnlyIn*. The ontology functions as a closed schema: during extraction, the model is restricted

to instantiating only predefined classes and properties, preventing uncontrolled relation generation or schema extension.

The document is subsequently parsed according to its legal hierarchy (3. Normative-Aware Segmentation). The caput is assigned a unique identifier and becomes the central node of the subgraph. Paragraphs inherit the same parent identifier, preserving their interpretative dependency on the main rule. Structural boundaries are strictly respected: neither the caput nor any paragraph is fragmented across chunks, ensuring that the semantic integrity of each normative unit is maintained.

Each structured segment is then submitted to the language model under ontology constraints (4. Ontology-Constrained LLM Triple Extraction). The LLM receives both the segmented text and the complete T-Box specification. From the example above, it generates triples representing that motorcycles are not permitted in the first-floor parking lot and that this restriction applies to residents and visitors, while parking is allowed only in the designated external area. In addition to these relational triples, the model instantiates a dedicated Rule node linked to the caput identifier. This node contains a synthesized textual formulation of the normative statement and connects to regulated entities and roles through typed predicates. By modeling the rule as a first-class entity, the graph captures the unified normative meaning rather than merely isolated binary relations.

Finally, the extracted triples are serialized and stored in a graph database (5. RDF Ingestion), while the corresponding structured text segments are embedded in a vector index under the same parent identifier. This dual ingestion enables hybrid retrieval: an informal query such as “Can I park my motorcycle in the downstairs garage?” may first retrieve semantically similar segments through embeddings, and then rely on the knowledge graph to identify the governing rule, its applicability conditions, and the spatial restriction explicitly encoded in the ontology.

This running example demonstrates how a compact normative excerpt is progressively transformed into a structured subgraph centered on a reified rule entity. By aligning abstraction (1), ontological constraint (2), hierarchical parsing (3), controlled extraction (4), and structured ingestion (5), the methodology preserves normative semantics and structural dependencies, rather than merely extracting disconnected entities and relations.

4. Experimental Evaluation

The evaluation aimed to assess two complementary aspects of the system: (i) the quality of retrieval and answer generation using the hybrid vector–graph approach, and (ii) the reliability of different automatic metrics for measuring factual correctness in a domain characterized by long, highly structured juridical content and user queries phrased in colloquial language. Because no public benchmark exists for condominium regulations, we constructed a synthetic dataset derived from an internal regulation document and used it to simulate real-world information needs.

4.1. Dataset Construction

The evaluation corpus was created from a complete condominium regulation document. Each chapter or rule served as a source text for generating a large set of natural-language questions. Using an LLM specifically instructed to produce informal, conversational phrasing, we synthesized 700 questions that mimic the style commonly observed in WhatsApp conversations between residents and condominium managers. Examples include questions containing hesitations, imprecise references, and narrative elements (“Então, tipo assim, eu comprei meu apartamento, mas ainda não me mudei... posso usar a piscina com meus amigos?”).

Each question was paired with a reference answer (the gold standard), drawn manually from the document. These references varied in length and detail: some were concise and direct, while others included longer excerpts from the regulation to capture nuances or exceptions. Together, these references form a challenging benchmark because the model must not only retrieve the correct segment but also summarize and interpret it accurately.

For model development, the dataset was divided into two subsets: an initial calibration set used for early validation and a main evaluation set consisting of 100 questions. All results reported in this section refer to the latter.

[Figure of gold standard line]

4.2. Experimental Procedure

Because the system incorporates two retrieval pathways, we evaluated them both independently and jointly. For each question, the system retrieved relevant chunks. These chunks were then provided to an LLM responsible for generating the final natural-language answer. The prompts used for vector-based answers and graph-based answers were kept as similar as possible, differing only in the additional structural information available through the KG.

The core task was to determine whether the system could recover the correct evidence and generate an answer semantically aligned with the gold standard. We evaluated two retrieval outcomes: whether the retrieved evidence was correct, and whether the final natural-language answer matched the reference in content and intention.

The output of each retrieval method therefore consisted of:

1. the selected chunk(s);
2. the short excerpt chosen by the LLM as justification;
3. the generated answer;
4. the predicates and neighbors retrieved from the KG (in case of graph method)

Metrics. To quantify similarity between generated answers and reference texts, we experimented with two categories of metrics: contextual token-level models (BERTScore [REF]) and embedding-based sentence-level models (cosine similarity).

BERTScore evaluates token overlap in a contextual embedding space and is used in text generation evaluation. However, its behavior proved problematic in our domain. Legal references often contain long, detailed normative statements, while a correct generated answer may condense this content into a short, direct sentence (“Sim, é permitido levar visitantes, mas o limite é de dois por apartamento”). In such cases, BERTScore penalizes the model heavily for omitting surface-level tokens, even when the semantic meaning matches. The metric also penalizes stylistic differences between formal juridical phrasing and colloquial answers.

For this reason, we also measured similarity using a sentence-level embedding model and computed cosine similarity between the generated answer and the gold standard. This metric proved more reliable in distinguishing correct from incorrect answers because it captures coarse semantic alignment rather than token-level correspondence. In practice, cosine similarity produced significantly higher agreement with human judgment, especially for short answers derived from lengthy articles.

4.3. Experimental Results

Across the 100-question evaluation set, vector retrieval achieved a cosine-similarity success rate of approximately 80%, while graph-based retrieval achieved 73%. These values indicate that both pathways are capable of recovering broadly correct evidence, with vector retrieval having a slight advantage due to its flexibility in handling informal questions. When considering cases where both methods surpassed the 0.75 cosine-similarity threshold, the intersection accounted for about 62% of the dataset, suggesting that in most instances both retrieval strategies converged on similar evidence.

When evaluated solely through BERTScore, the results were considerably weaker, with only 28% of vector-based answers and 18% of graph-based answers scoring above the commonly accepted F1 threshold of 0.75. In many of these cases, inspection confirmed that the system’s answer was factually correct but penalized for paraphrasing, shortening, or rephrasing the original regulation. These observations reinforce the inadequacy of token-level metrics in domains where answers must translate dense formal language into concise, conversational guidance.

Although graph retrieval did not outperform vector retrieval in raw similarity metrics, its principal value emerged in the explainability layer. Answers generated through graph retrieval contained structured evidence in the form of chunk identifiers and associated legal relationships, which provided transparency into why the rule applied. In several cases where the vector-based answer appeared correct but incomplete, the graph-based explanation exposed relevant applicability conditions, such as restrictions on permitted hours, mandatory presence of residents, or distinctions between types of visitors. This additional structure is especially important for legal and quasi-legal contexts, where users often require justification rather than merely a correct conclusion.

5. Discussion

The results of this study highlight both the potential and the challenges of combining Knowledge Graphs and vector retrieval for answering juridical questions derived from condominium regulations. While the hybrid architecture provides retrieval performance and enhances explainability, it also introduces layers of complexity that require careful consideration when applied to real-world contexts.

A benefit of the KG component lies in its capacity to make explicit the semantic structure inherent in juridical documents. Regulations frequently articulate permissions, obligations, exceptions, and applicability conditions that depend on relationships between entities such as residents, service providers, common areas, and specific articles or clauses. Vector search can approximate these relationships through distributional similarity, but it cannot represent them explicitly. In contrast, the KG encodes these connections directly, enabling the system to surface information about which rule governs a particular situation, what entity it applies to, or how it interacts with other rules. This structural grounding increases user trust, particularly in legal scenarios where the validity of an answer depends on correctness and verifiable justification.

However, constructing and maintaining a Knowledge Graph for juridical content is more demanding than relying solely on embeddings. The ingestion pipeline must guarantee adherence to the ontology, since even minor inconsistencies can propagate through inference and produce misleading relationships. The need to serialize LLM-extracted graphs into RDF/Turtle format is a direct response to these concerns, as is the necessity of curating the ontology itself to ensure that it reflects the nuances of normative language. These requirements underscore an inherent trade-off: while KG offer richer and more interpretable structures, their creation introduces strict constraints on schema design, extraction accuracy, and ingestion fidelity.

Another important observation emerges from the evaluation metrics. The inadequacy of token-level metrics such as BERTScore suggests that many standard NLP evaluation tools are poorly aligned with the needs of juridical or regulatory retrieval systems. Answers in such domains are expected to be short, decisive, and framed in plain language, whereas the gold-standard references often contain lengthy passages embedded in formal legal syntax. The mismatch between these linguistic registers leads to artificially low scores even when the content is correct. Embedding-based metrics, while more tolerant of paraphrasing, still have limitations; they measure semantic proximity but do not explicitly evaluate factual grounding or logical correctness. This opens space for alternative evaluation strategies, including LLM-as-a-judge approaches such as RAGAS, which may better capture the alignment between retrieved evidence, generated reasoning, and the final answer.

Although the Knowledge Graph did not outperform vector retrieval in terms of raw accuracy, its contributions to explainability are significant. By surfacing predicates, neighbor entities, and class hierarchies, the graph provides a pathway for users to inspect and verify the normative basis of an answer. This becomes important when dealing with temporal validity, amendments to regulations, or conflicts between documents. For instance, identifying whether a rule in a convention has been superseded by a newer assembly decision is inherently a graph reasoning problem, not a vector matching problem. The current version of the system does not yet exploit these capabilities fully, but the structure is in place to support them.

Looking forward, several avenues for improvement emerge. From the perspective of the KG, there is

considerable room to expand the ontology with temporal qualifiers, document versioning properties, and richer representations of exceptions or conditional clauses. Such additions would enable more precise reasoning about when a rule applies and how its applicability changes over time.

Another direction concerns the triple extraction process. Although the schema-restricted prompting strategy reduced hallucinations, it remains sensitive to chunk boundaries and to ambiguities within the text. Future work may incorporate multi-pass extraction, cross-chunk aggregation, or confidence calibration to further improve the stability of extracted triples. Similarly, integrating a validation layer could help detect inconsistencies or missing relationships before ingestion.

Finally, evaluation remains an open challenge. Embedding-based metrics provide a usable but imperfect solution, and future studies should consider incorporating human evaluation, LLM-based judgment, or task-specific scoring frameworks that better reflect the requirements of juridical decision support. A promising line of research involves evaluating the explanation rather than only the final answer.

In summary, the hybrid architecture presented here shows that KGs and vector retrieval complement each other in meaningful ways. While vector search provides accessibility and linguistic robustness, the KG provides interpretability and structured reasoning important for trustworthy domain-specific applications. The framework lays the groundwork for more advanced capabilities, including temporal reasoning, normative conflict detection, and richer explainability mechanisms, which will be explored in future work.

6. Conclusion

References

- [1] Y. Zhu, X. Wang, J. Chen, S. Qiao, Y. Ou, Y. Yao, S. Deng, H. Chen, N. Zhang, Llms for knowledge graph construction and reasoning: Recent capabilities and future opportunities, 2024. URL: <https://arxiv.org/abs/2305.13168>. `arXiv:2305.13168`.
- [2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph rag approach to query-focused summarization, 2025. URL: <https://arxiv.org/abs/2404.16130>. `arXiv:2404.16130`.
- [3] H. de Martim, An ontology-driven graph rag for legal norms: A structural, temporal, and deterministic approach, 2025. URL: <https://arxiv.org/abs/2505.00039>. `arXiv:2505.00039`.
- [4] M. Fernández-López, A. Gómez-Pérez, N. Juristo, Methontology: From ontological art towards ontological engineering, 1997. AAAI Technical Report.